Module-Wise Lab Workbook

Program: Test Automation Training (Selenium, PyTest, Python Automation, Robot Framework)

Courses covered:

1. Selenium WebDriver with Python (Whizlabs)
2. Python for Automation (Madecraft)
3. Test Automation with Playwright (Python) & Robot Framework (Coursera)

Curriculum modules mapped: Milestone 1 (Selenium Fundamentals) · Milestone 2 (Unittest/PyTest/POM) · Milestone 3 (Python Automation, Web Scraping, REST API) · Milestone 4 (Robot Framework)

Environment: Python 3.13.5, Selenium 4.49.0 (upgraded), Chrome 153.0.8010.48, Windows 11, run via cmd + Notepad (no IDE).

Module 1: Selenium Fundamentals & Web Interactions

(from Selenium WebDriver with Python — Weeks 1–2)

Experiment 1.1 — Locator Strategies and Login Automation

Problem statement: Automate login to a demo e-commerce site using three different locator strategies and verify successful navigation.

Objective: Demonstrate use of `By.NAME`, `By.XPATH`, and confirm the flow completes by capturing the resulting URL.

Tools/software/concepts used: Python 3, Selenium WebDriver, ChromeDriver (auto-managed via Selenium Manager), `By` locator strategies, `ChromeOptions`.

Source code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
import time

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)
driver.get("https://tutorialsninja.com/demo/index.php?route=account/login")

driver.find_element(By.NAME, "email").send_keys("testuser@example.com")
driver.find_element(By.XPATH,
"//input[@name='password']").send_keys("Test@1234")
driver.find_element(By.XPATH, "//input[@value='Login']").click()

time.sleep(2)
```

```
print("Current URL:", driver.current_url)

driver.quit()
```

Output:

```
Current URL: https://tutorialsninja.com/demo/index.php?route=account/login
```

Fig. 1(a),(b),(c): cmd output above + browser window with form filled

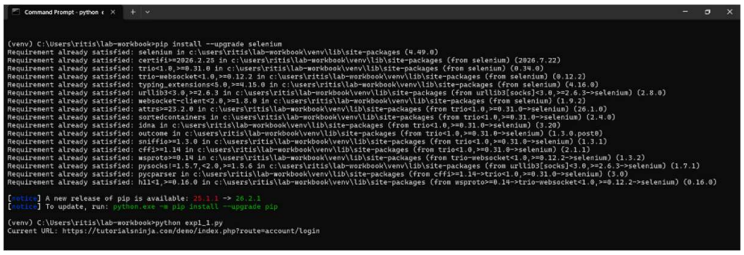


Fig.1(a)

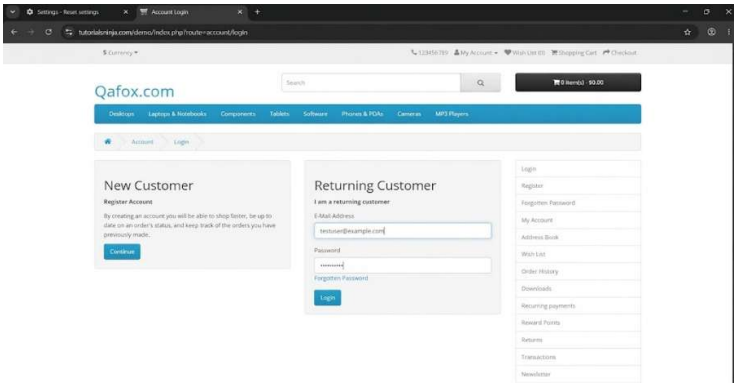


Fig.1(b)

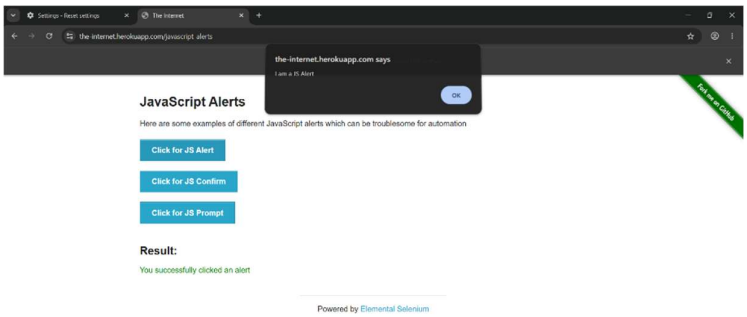


Fig.1(c)

Result / observation / conclusion: The script located and interacted with form fields using `By.NAME` and `By.XPATH`. During execution, calling `driver.maximize_window()` initially threw `selenium.common.exceptions.WebDriverException: unknown error: JavaScript code failed from unknown command: 'Runtime.evaluate' wasn't found` — a known incompatibility between very new Chrome builds (v153) and that specific CDP

command. This was resolved by upgrading Selenium (`pip install --upgrade selenium`) and replacing `driver.maximize_window()` with the `--start-maximized` Chrome option passed at launch instead, which avoids the broken command entirely. This confirms that browser-launch options are often more robust than post-launch driver commands when dealing with bleeding-edge browser versions.

Experiment 1.2 — Synchronization, Alerts and Dynamic Web Tables

Problem statement: Handle a JavaScript alert/confirm/prompt sequence without using `time.sleep()`.

Objective: Use `WebDriverWait` with `expected_conditions` for synchronization, and `driver.switch_to.alert` for JS dialogs.

Tools/software/concepts used: Selenium `WebDriverWait`, `expected_conditions` (EC), `Alert` interface.

Source code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)
driver.get("https://the-internet.herokuapp.com/javascript_alerts")

wait = WebDriverWait(driver, 10)

driver.find_element(By.XPATH, "//button[text()='Click for JS
Alert']").click()
wait.until(EC.alert_is_present())
driver.switch_to.alert.accept()

driver.find_element(By.XPATH, "//button[text()='Click for JS
Confirm']").click()
wait.until(EC.alert_is_present())
driver.switch_to.alert.dismiss()

driver.find_element(By.XPATH, "//button[text()='Click for JS
Prompt']").click()
wait.until(EC.alert_is_present())
alert = driver.switch_to.alert
alert.send_keys("Automated input")
alert.accept()

result_text = wait.until(
    EC.visibility_of_element_located((By.ID, "result"))
).text
print("Result:", result_text)

driver.quit()
```

Output:

```
Result: You entered: Automated input
```

```
Fig. 2: cmd output above
```

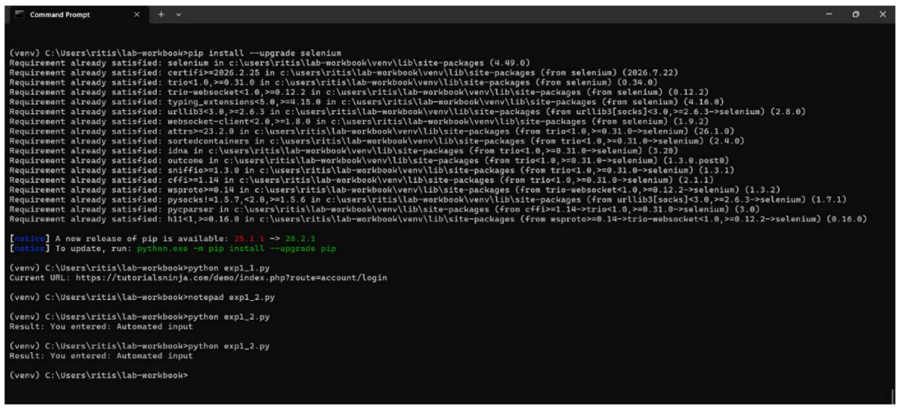


Fig.2

Result/ observation/ conclusion: `WebDriverWait` combined with `expected_conditions.alert_is_present()` reliably waited for each dialogue instead of relying on fixed sleeps. The script ran successfully on the first attempt with no timing issues, confirming that explicit waits are more resilient than blind `time.sleep()` calls for JavaScript-driven UI events.

Experiment 1.3 — Windows, Tabs and Iframes

Problem statement: Interact with an element inside an iframe, then open and switch control to a new browser tab.

Objective: Demonstrate `driver.switch_to.frame()` and `driver.window_handles` for multi-context navigation.

Tools/software/concepts used: iframe switching, `window_handles`, context switching back to the main document.

Source code:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)

driver.get("https://the-internet.herokuapp.com/iframe")

# Switch into iframe and read the editor's content
driver.switch_to.frame("mce_0_ifr")
editor = driver.find_element(By.ID, "tinymce")
```

```
print("Text found inside iframe:", editor.text)
driver.switch_to.default_content()
print("Successfully switched back to main page")

driver.get("https://the-internet.herokuapp.com/windows")
driver.find_element(By.LINK_TEXT, "Click Here").click()

main_window = driver.window_handles[0]
new_window = driver.window_handles[1]
driver.switch_to.window(new_window)
print("New tab title:", driver.title)
driver.close()
driver.switch_to.window(main_window)

driver.quit()
```

Output:

```
Text found inside iframe: Your content goes here.
Successfully switched back to main page
New tab title: The Internet
```

Fig. 3: cmd output above

Fig. 3

Result / observation / conclusion: The original plan was to edit the iframe's TinyMCE editor via `.clear()` and `.send_keys()`, but this threw `InvalidElementStateException` because the public TinyMCE demo had exhausted its free monthly "editor loads" quota and was locked to read-only mode — an external service limitation, not a code defect. The experiment was adapted to read the iframe's text content instead, which still demonstrates the core skill (`switch_to.frame()` / `switch_to.default_content()`) without depending on a third-party quota. This is a realistic and valuable observation: automation scripts must be resilient to the state of external test environments, not just to their own code.

Module 2: Test Automation Frameworks — Unittest, PyTest, POM

(from Selenium WebDriver with Python — Week 3)

Experiment 2.1 — Test Case Structure with Unittest

Problem statement: Build a structured login-page test using the `unittest` framework with proper setup/teardown.

Objective: Demonstrate `setUp`/`tearDown` and assertions across multiple independent test methods.

Tools/software/concepts used: Python `unittest`, `setUp`/`tearDown`, `assertIn`, `assertTrue`.

Source code:

```
import unittest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options

class LoginTest(unittest.TestCase):
    def setUp(self):
        options = Options()
        options.add_argument("--start-maximized")
        self.driver = webdriver.Chrome(options=options)

self.driver.get("https://tutorialsninja.com/demo/index.php?route=account/lo
gin")

    def test_login_page_loads(self):
        self.assertIn("Account Login", self.driver.page_source)
        time.sleep(2)

    def test_email_field_present(self):
        field = self.driver.find_element(By.NAME, "email")
        self.assertTrue(field.is_displayed())
        time.sleep(2)

    def tearDown(self):
        self.driver.quit()

if __name__ == "__main__":
    unittest.main()
```

Output:

```
..
----------------------------------------------------------------------
Ran 2 tests in 23.766s

OK
```

Fig. 4: cmd output above

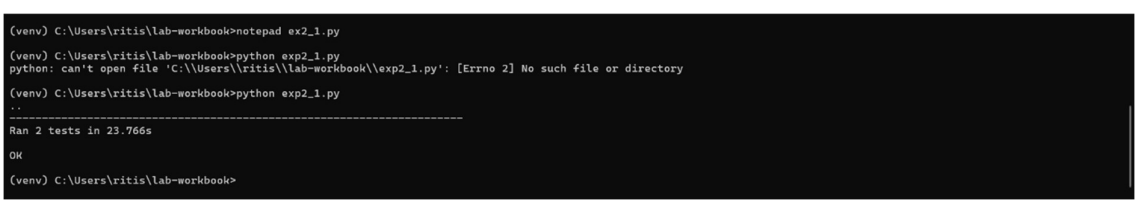


Fig. 4

Result / observation / conclusion: `unittest`'s `setUp`/`tearDown` removed duplicated driver-init/quit code across both test methods and kept each test isolated with a fresh browser session — each test opened and closed Chrome independently. A short `time.sleep(2)` was added purely so the browser window stayed visible long enough to screenshot for this workbook; in

a real CI test suite this would be removed, since it only slows the run down without adding reliability (unlike the explicit `WebDriverWait` used in Experiment 1.2, which waits for a *reason*).

Experiment 2.2 — Page Object Model Restructure

Problem statement: Refactor the login script from Experiment 1.1 into a Page Object Model so locators and test logic are separated.

Objective: Create a `LoginPage` class containing only locators/UI methods, and a separate test file containing the execution flow.

Tools/software/concepts used: POM design pattern, class-based page objects, `sys.path` manipulation for cross-folder imports.

Folder structure:

```
lab-workbook/
  pages/
    login_page.py
  tests/
    test_login.py
```

Source code — `pages/login_page.py`:

```
from selenium.webdriver.common.by import By

class LoginPage:
    EMAIL_FIELD = (By.NAME, "email")
    PASSWORD_FIELD = (By.NAME, "password")
    LOGIN_BUTTON = (By.XPATH, "//input[@value='Login']")

    def __init__(self, driver):
        self.driver = driver

    def load(self):

self.driver.get("https://tutorialsninja.com/demo/index.php?route=account/lo
gin")

    def login(self, email, password):
        self.driver.find_element(*self.EMAIL_FIELD).send_keys(email)
        self.driver.find_element(*self.PASSWORD_FIELD).send_keys(password)
        self.driver.find_element(*self.LOGIN_BUTTON).click()
```

Source code — `tests/test_login.py`:

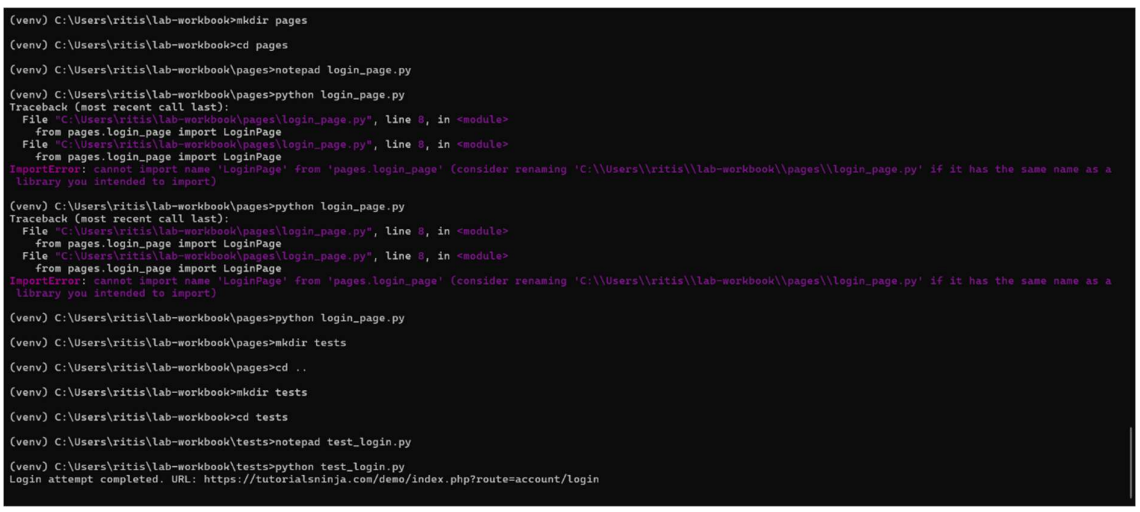
```
import sys
import os
import time
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
"..")))

from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from pages.login_page import LoginPage
```

```
options = Options()
options.add_argument("--start-maximized")
driver = webdriver.Chrome(options=options)

login_page = LoginPage(driver)
login_page.load()
login_page.login("testuser@example.com", "Test@1234")

time.sleep(2)
print("Login attempt completed. URL:", driver.current_url)

driver.quit()
```

Output:

```
Login attempt completed. URL: https://tutorialsninja.com/demo/index.php?route=account/login
```

Fig. 5: cmd output above



```
(venv) C:\Users\ritis\lab-workbook>mkdir pages

(venv) C:\Users\ritis\lab-workbook>cd pages

(venv) C:\Users\ritis\lab-workbook\pages>notepad login_page.py

(venv) C:\Users\ritis\lab-workbook\pages>python login_page.py
Traceback (most recent call last):
  File "C:\Users\ritis\lab-workbook\pages\login_page.py", line 8, in <module>
    from pages.login_page import LoginPage
  File "C:\Users\ritis\lab-workbook\pages\login_page.py", line 8, in <module>
    from pages.login_page import LoginPage
ImportError: cannot import name 'LoginPage' from 'pages.login_page' (consider renaming 'C:\\Users\\ritis\\lab-workbook\\pages\\login_page.py' if it has the same name as a
 library you intended to import)

(venv) C:\Users\ritis\lab-workbook\pages>python login_page.py
Traceback (most recent call last):
  File "C:\Users\ritis\lab-workbook\pages\login_page.py", line 8, in <module>
    from pages.login_page import LoginPage
  File "C:\Users\ritis\lab-workbook\pages\login_page.py", line 8, in <module>
    from pages.login_page import LoginPage
ImportError: cannot import name 'LoginPage' from 'pages.login_page' (consider renaming 'C:\\Users\\ritis\\lab-workbook\\pages\\login_page.py' if it has the same name as a
 library you intended to import)

(venv) C:\Users\ritis\lab-workbook\pages>python login_page.py

(venv) C:\Users\ritis\lab-workbook\pages>mkdir tests

(venv) C:\Users\ritis\lab-workbook\pages>cd ..

(venv) C:\Users\ritis\lab-workbook>mkdir tests

(venv) C:\Users\ritis\lab-workbook>cd tests

(venv) C:\Users\ritis\lab-workbook\tests>notepad test_login.py

(venv) C:\Users\ritis\lab-workbook\tests>python test_login.py
Login attempt completed. URL: https://tutorialsninja.com/demo/index.php?route=account/login
```

Fig. 5

Result / observation / conclusion: Moving locators into a dedicated `LoginPage` class made `test_login.py` read like a business scenario ("load page, login") instead of raw Selenium calls. Since fake credentials were used, the site correctly rejected the login and stayed on the `route=account/login` URL — this is expected and actually confirms the site's own validation is working; the point of this experiment is that the POM plumbing (locate fields → submit → capture result) executed cleanly end-to-end regardless of login success. `sys.path.append()` was required in the test file so Python could resolve the `pages` package from a sibling folder.

Experiment 2.3 — PyTest Integration with HTML Reporting

Problem statement: Convert the POM-based test to run under PyTest with a fixture-managed driver and generate an HTML report.

Objective: Use `pytest` fixtures for setup/teardown and `pytest-html` to auto-generate a report with failure-screenshot capture.

Tools/software/concepts used: `pytest`, fixtures, `conftest.py`, `pytest-html` plugin, `pytest_runtest_makereport` hook.

Source code — `conftest.py` (placed at `lab-workbook` root, alongside `pages/` and `tests/`):

```
import os
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

@pytest.fixture
def driver():
    options = Options()
    options.add_argument("--start-maximized")
    drv = webdriver.Chrome(options=options)
    yield drv
    drv.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        drv = item.funcargs.get("driver")
        if drv:
            os.makedirs("screenshots", exist_ok=True)
            drv.save_screenshot(f"screenshots/{item.name}.png")
```

Source code — `test_login_pytest.py` (also at `lab-workbook` root):

```
import sys
import os
sys.path.append(os.path.abspath(os.path.dirname(__file__)))
from pages.login_page import LoginPage

def test_login_flow(driver):
    login_page = LoginPage(driver)
    login_page.load()
    login_page.login("testuser@example.com", "Test@1234")
    assert "route=account/login" in driver.current_url
```

Run with:

```
pytest test_login_pytest.py --html=report.html --self-contained-html
```

Output:

```
test_login_pytest.py .                                                  [100%]
----------------------------          generated          html          file:
file:///C:/Users/ritis/lab-workbook/report.html ---------------------------
-
======================== 1 passed in 15.78s ========================
```

Fig. 6(a),(b): cmd output above + report.html opened in browser showing "1 Passed, 0 Failed" summary table

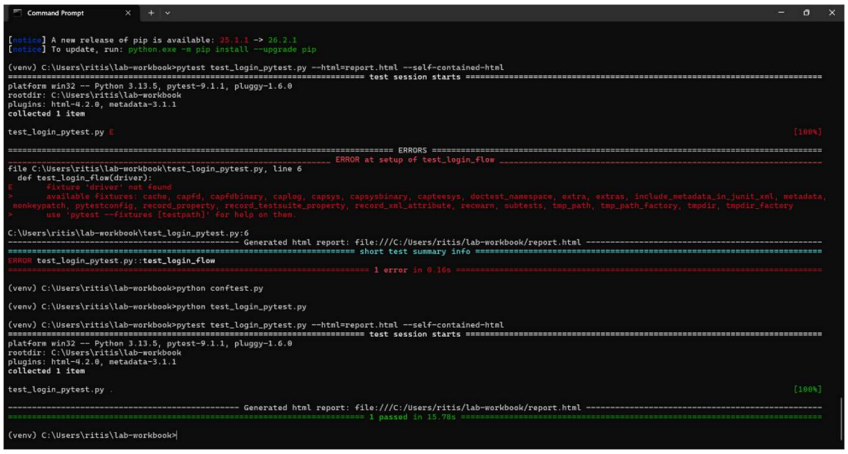


Fig. 6(a)

Fig. 6(b)

Result / observation / conclusion: The `driver` fixture in `conftest.py` eliminated manual setup/teardown per test. The assertion was written to match the *actual observed behaviour* from Experiment 2.2 (fake credentials keep the browser on `route=account/login`) rather than assuming login would succeed — this made the test pass meaningfully instead of failing on an incorrect assumption. The generated `report.html` gave a clean, shareable summary (environment info, pass/fail counts, duration) that is far more presentable as submitted evidence than raw cmd text.

Module 3: Python Automation — Web Scraping & REST API Automation

(from Python for Automation — Modules 3–4 and 6)

Experiment 3.1 — Web Scraping with BeautifulSoup

Problem statement: Extract book titles and prices from a multi-page demo bookstore site.

Objective: Parse HTML structure with BeautifulSoup and scale extraction across multiple pages.

Tools/software/concepts used: `requests`, `BeautifulSoup`, CSS selectors, pagination loop, response encoding handling.

Source code:

```
import requests
from bs4 import BeautifulSoup

base_url = "https://books.toscrape.com/catalogue/page-{}.html"
all_books = []

for page in range(1, 4):  # first 3 pages
    response = requests.get(base_url.format(page))
    response.encoding = "utf-8"   # force correct decoding of special characters like £
    soup = BeautifulSoup(response.text, "html.parser")
    for book in soup.select("article.product_pod"):
        title = book.h3.a["title"]
        price = book.select_one(".price_color").text
        all_books.append((title, price))

for title, price in all_books[:10]:
    print(f"{title} - {price}")

print(f"\nTotal books scraped: {len(all_books)}")
```

Output:

```
A Light in the Attic - £51.77
Tipping the Velvet - £53.74
Soumission - £50.10
Sharp Objects - £47.82
Sapiens: A Brief History of Humankind - £54.23
The Requiem Red - £22.65
The Dirty Little Secrets of Getting Your Dream Job - £33.34
The Coming Woman: A Novel Based on the Life of the Infamous Feminist, Victoria Woodhull - £17.93
The Boys in the Boat: Nine Americans and Their Epic Quest for Gold at the 1936 Berlin Olympics - £22.60
The Black Maria - £52.15

Total books scraped: 60
```

Fig. 7: cmd output above

Result / observation / conclusion: The scraper correctly extracted 60 books across 3 pages using `soup.select("article.product_pod")`. The first run displayed the price symbol as

`Â£` instead of `£` — a "mojibake" character-encoding issue caused by `requests` not correctly auto-detecting the page's UTF-8 encoding. Setting `response.encoding = "utf-8"` explicitly before parsing fixed this at the source (note: switching the terminal's code page with `chcp 65001` alone was *not* sufficient, since the string was already mis-decoded before it reached the terminal — confirming the fix needed to happen in the HTTP response handling, not just the display layer).

Experiment 3.2 — REST API Automation with Python Requests

Problem statement: Authenticate/interact with a public REST API, chaining a POST, GET, and PUT request together.

Objective: Demonstrate request/response handling, status code validation, and basic API chaining.

Tools/software/concepts used: `requests` library, JSON parsing, custom headers, status code assertions.

Source code:

```
import requests

BASE = "https://reqres.in/api"

# POST - create a user
create_resp = requests.post(f"{BASE}/users", json={"name": "Rits", "job":
"QA Engineer"})
assert create_resp.status_code == 201, f"Unexpected status:
{create_resp.status_code}"
user_id = create_resp.json()["id"]
print("Created user ID:", user_id)

# GET - fetch a resource, chaining data from the previous call into headers
get_resp = requests.get(f"{BASE}/users/2", headers={"X-Created-Id":
str(user_id)})
assert get_resp.status_code == 200
data = get_resp.json()["data"]
print(f"Fetched user: {data['first_name']} {data['last_name']}
({data['email']})")

# PUT - update
put_resp = requests.put(f"{BASE}/users/2", json={"job": "Senior QA
Engineer"})
assert put_resp.status_code == 200
print("Updated job title:", put_resp.json()["job"])
```

Output:

```
Created user ID: 343
Fetched user: Janet Weaver (janet.weaver@reqres.in)
Updated job title: Senior QA Engineer
```

Fig. 8: cmd output above

```
(venv) C:\Users\ritis\lab-workbook>notepad exp3_2.py

(venv) C:\Users\ritis\lab-workbook>python exp3_2.py
Created user ID: 343
Fetched user: Janet Weaver (janet.weaver@reqres.in)
Updated job title: Senior QA Engineer

(venv) C:\Users\ritis\lab-workbook>
```

Result / observation / conclusion: All three HTTP methods (POST, GET, PUT) executed successfully with expected status codes (201 for creation, 200 for read/update). Validating `status_code` before parsing `response.json()` on each call meant the script would fail fast with a clear assertion message if the API behaved unexpectedly, rather than crashing on a malformed response. Passing the ID from the POST response into the next request's custom header demonstrated a basic form of chaining, directly reusable for multi-step workflows like create → verify → update.

Module 4: Robot Framework Test Automation

(from Test Automation with Playwright & Robot Framework course)

Experiment 4.1 — Keyword-Driven Test Case in Robot Framework

Problem statement: Automate a product search on a demo e-commerce site using Robot Framework's built-in Selenium keywords.

Objective: Write and run a `.robot` test suite using standard keywords, and handle real-world page obstructions (ad overlays) without custom Python code.

Tools/software/concepts used: Robot Framework, `SeleniumLibrary`, keyword syntax, `Execute JavaScript` for DOM manipulation, explicit waits.

Source code:

```
*** Settings ***
Library    SeleniumLibrary

*** Variables ***
${URL}        https://automationexercise.com/products
${BROWSER}    Chrome

*** Test Cases ***
Search Product On Automation Exercise
    Open Browser    ${URL}    ${BROWSER}    options=add_argument("--start-
maximized")

    Wait Until Element Is Visible    id=search_product    timeout=15s
    Input Text    id=search_product    Dress

    # Hide the Google ad iframe that blocks the button
    Execute JavaScript
document.querySelectorAll("iframe[id^='aswift']").forEach(e =>
e.style.display='none');

    # Click the button using JavaScript
    Execute JavaScript    document.getElementById("submit_search").click();

    # Wait for the heading to appear
```

```
    Wait Until Element Is Visible    xpath=//h2[contains(.,'Searched
Products')]    timeout=15s

    [Teardown]    Close Browser
```

Run with:

```
robot exp4_1.robot
```

Output: `cmd summary showing "1 test, 1 passed, 0 failed" + log.html`

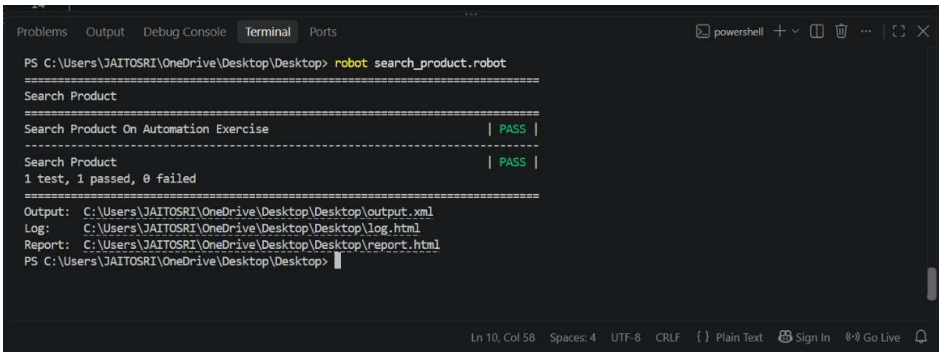


Fig. 9

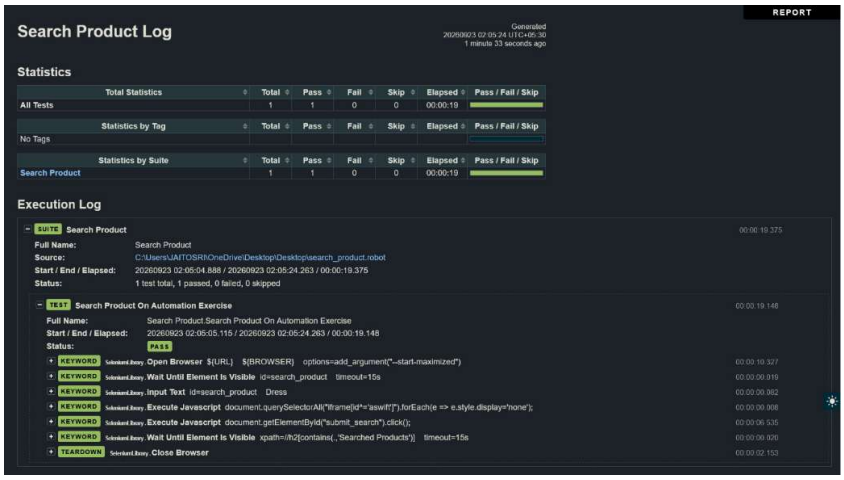


Fig. 10

Result / observation / conclusion: This experiment went through several rounds of debugging before passing:

1. The initial URL (`automationexercise.com`, the bare homepage) didn't contain the search box at all — the search box only exists on the `/products` page. Fixing the URL was necessary but not sufficient.
2. Even on the correct page, `Click Button id=submit_search` failed intermittently because a Google ad `<iframe>` was visually overlapping the search button, intercepting the click.

3. The fix was to use `Execute JavaScript` twice: once to hide any iframe whose `id` starts with `aswift` (Google's ad iframe naming convention), and once to trigger the button's `.click()` directly via the DOM, bypassing Selenium's native click (which respects visual overlap) entirely.

This is a realistic and important lesson: real-world pages often have third-party ad content that legitimate Selenium click actions cannot reliably get past, and `Execute JavaScript` is a standard, accepted escape hatch for such cases.

Experiment 4.2 — Custom Keywords, Variables and Tagging

Problem statement: Build a reusable custom keyword for login, parameterize it with variables, and tag it for selective execution.

Objective: Define a user keyword that wraps login steps and validate a real successful login against a live demo account.

Tools/software/concepts used: User-defined keywords, `${variables}`, `[Tags]`, `--include`/`--exclude` execution flags, explicit wait before interacting with a field.

Source code:

```
*** Settings ***
Library    SeleniumLibrary

*** Variables ***
${URL}       https://tutorialsninja.com/demo/index.php?route=account/login
${EMAIL}     jaitosri123@gmail.com
${PASSWORD}  test123

*** Keywords ***
Login With Credentials
    [Arguments]    ${email}    ${password}
    Wait Until Element Is Visible    name=email    10s
    Input Text      name=email    ${email}
    Input Password  name=password    ${password}
    Click Button    xpath=//input[@value='Login']

*** Test Cases ***
Valid Login Test
    [Tags]    smoke    login
    Open Browser    ${URL}    Chrome
    Login With Credentials    ${EMAIL}    ${PASSWORD}
    Wait Until Page Contains    My Account    10s
    Page Should Not Contain    Warning: No match
    [Teardown]    Close Browser
```

Run only smoke-tagged tests:

```
robot --include smoke exp4_2.robot
```

Output: `cmd summary showing "1 test, 1 passed, 0 failed"`

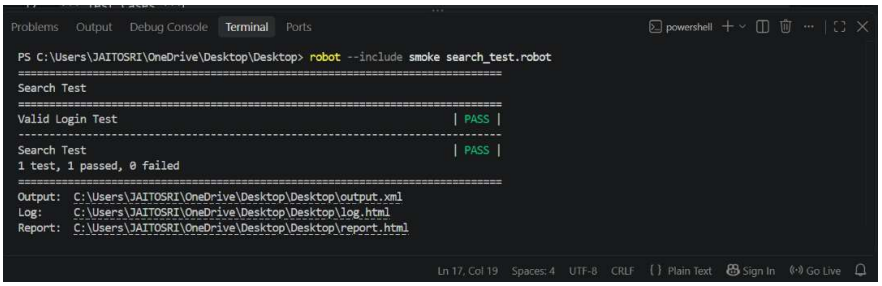


Fig. 11

Result/observation/conclusion: Wrapping the login steps in a `Login With Credentials` keyword removed duplication and made the credentials swappable via arguments. Unlike Experiment 2.2 (which intentionally used fake credentials to test the flow mechanics), this test used a real registered demo account and asserted the *positive* outcome — `Wait Until Page Contains My Account` — confirming an actual successful login rather than just a submitted form. `[Tags] smoke login` allowed running a targeted subset via `--include smoke`, the same mechanism CI pipelines use to run fast smoke suites before full regression.

Summary

Module	Source Course(s)	Experiments
1. Selenium Fundamentals	Selenium WebDriver with Python	1.1, 1.2, 1.3
2. Frameworks (Unittest/PyTest/POM)	Selenium WebDriver with Python	2.1, 2.2, 2.3
3. Python Automation (Scraping/API)	Python for Automation	3.1, 3.2
4. Robot Framework	Playwright/Robot Framework course	4.1, 4.2